

AI Software Development Internship

Week 3 — Detailed Task Sheet

SQL Server & Entity Framework Core • API/Angular Integration
Git & GitHub Workflow • AI & Python Foundations

Tracks: C# / .NET, Angular, and AI (Python)

Participants: Two Intern Candidates

Program length: 8 weeks total | This document: Week 3 of 8

Where we left off

Weeks 1 and 2 covered C#/.NET fundamentals, OOP, collections, LINQ, ASP.NET Core basics, layered architecture, and Angular components/forms. Basic table creation and record insertion were already practiced in Weeks 1–2, so this week does not repeat that — it moves straight into relational design, Entity Framework Core, wiring the real database into the API, connecting Angular to that live API, a proper Git/GitHub workflow, and the first hands-on AI/Python script.

How to Use This Document

Every part below follows the same four stages used in Weeks 1 and 2, with one addition — a Git checkpoint at the end of nearly every stage, since from this week onward all work must be committed and pushed as it is completed, not in one large commit at the end.

- **1. Learn** — Read the explanation given here. It tells you both *what* the concept is and *why* it exists, not just a term to memorize.
- **2. Practice** — Small, isolated exercises for one concept at a time. Do not skip these to jump to the project.
- **3. Git Checkpoint** — Commit and push the practice work with a proper commit message before moving on.
- **4. Project** — Apply everything together in the Week 3 Library project.
- **5. Demonstrate** — Explain the work to a mentor and walk through the GitHub repository, the commit history, and at least one Pull Request.

Important rule (same as Weeks 1–2)

Do not move to the project just because the calendar says so. A candidate should be able to explain a concept out loud, without looking at notes, before using it in the project.

Daily pattern: Study 1–2 hours → Code 2–3 hours → Review 30–45 minutes → Commit and push to GitHub before ending the day. From this week on, "commit at the end of the day" becomes "commit after every small working change," which is explained in Part F.

PART A — RELATIONAL DATABASE DESIGN (BEYOND BASIC TABLES)

You already know how to create a table and insert/select/update/delete rows from Weeks 1–2. This part goes one level deeper: **designing** tables that relate to each other correctly, and querying across them. This is what the Library project actually needs, since a Book belongs to an Author and a Category.

What to learn

- **Normalization (concept level only)** — why data is split across multiple related tables instead of one giant table (avoids repeating an author's name in every single book row).
- **Primary key** — the column that uniquely identifies a row (e.g. BookId).
- **Foreign key** — a column in one table that points to the primary key of another table (e.g. AuthorId inside the Books table).
- **Relationship types** — one-to-many (one Author has many Books), many-to-many (a Book can have many Categories, and a Category can apply to many Books).
- **JOIN** — combining rows from two or more tables based on a matching key, so you can ask "show me each book together with its author's name" in a single query.
- **Indexes (awareness only)** — why a primary key is automatically fast to search, and what an index does in general terms.

How to do it

Use SQL Server Management Studio (SSMS) or Azure Data Studio, connected to your local SQL Server instance (the same one used in Weeks 1–2). Create a new database called LibraryDb_Week3 for this week's practice so it stays separate from any old test data.

```
CREATE DATABASE LibraryDb_Week3;  
GO  
USE LibraryDb_Week3;  
GO
```

Practice — do these before the project

- Create an Authors table (AuthorId PK, FullName) and a Books table (BookId PK, Title, AuthorId FK referencing Authors).
- Insert 3 authors and 6 books split between them.
- Write a JOIN query that lists every book together with its author's name in one result set.
- Create a Categories table and a link table BookCategories (BookId, CategoryId) to represent many-to-many, then insert sample rows connecting books to more than one category.
- Write a query that returns all categories for one specific book using the link table.
- Try deleting an Author that still has Books linked to it and read the error SQL Server gives you — explain in your own words why it refused.

```
SELECT b.Title, a.FullName AS Author  
FROM Books b  
JOIN Authors a ON b.AuthorId = a.AuthorId  
ORDER BY a.FullName;
```

Git Checkpoint

Save your .sql scripts (table creation + sample inserts + the JOIN queries) inside a /sql folder in your repository and commit them: `git add sql/ && git commit -m "feat: add Week 3 relational schema and sample data"`

PART B — ENTITY FRAMEWORK CORE

EF Core lets your C# code talk to the database using C# classes and LINQ instead of writing raw SQL by hand for every operation. This part is the heart of Week 3.

What to learn

- **EF Core's purpose** — it maps a C# class (e.g. `Book`) to a database table, so `Books.Add(newBook)` becomes an INSERT statement behind the scenes.
- **DbContext** — the class that represents your connection/session to the database.
- **DbSet<T>** — a property on the `DbContext` representing one table (e.g. `DbSet<Book> Books`).
- **Entity models** — the plain C# classes with properties that EF Core maps to columns, plus navigation properties that represent relationships (e.g. a `Book` has an `Author` `Author` property).
- **Connection strings** — how the app knows which database/server to connect to, stored in `appsettings.json`, never hard-coded in a class.
- **Migrations** — a versioned, generated script that changes the real database schema to match your C# models, so the database structure evolves the same way your code does.
- **SaveChangesAsync** — the call that actually sends your pending Add/Update/Delete changes to the database.

How to do it — step by step

- 1 In your ASP.NET Core Web API project, install the EF Core packages: `dotnet add package Microsoft.EntityFrameworkCore.SqlServer` and `dotnet add package Microsoft.EntityFrameworkCore.Tools`.
- 2 Create entity classes `Book.cs`, `Author.cs`, and `Category.cs` in a `/Models` folder.
- 3 Create a `LibraryDbContext.cs` in a `/Data` folder that inherits from `DbContext` and exposes a `DbSet` for each entity.
- 4 Add the connection string to `appsettings.json` (and a separate, git-ignored `appsettings.Development.json` for local secrets).
- 5 Register the `DbContext` in `Program.cs` using `builder.Services.AddDbContext<LibraryDbContext>(...)` so it can be injected into your services via Dependency Injection.
- 6 Generate the first migration and apply it to create the real tables.

```
// Models/Book.cs
public class Book
{
    public int BookId { get; set; }
    public string Title { get; set; } = string.Empty;
    public int AuthorId { get; set; }
    public Author? Author { get; set; }
}

// Data/LibraryDbContext.cs
public class LibraryDbContext : DbContext
{
    public LibraryDbContext(DbContextOptions<LibraryDbContext> options) : base(options) { }
```

```
public DbSet<Book> Books => Set<Book>();  
public DbSet<Author> Authors => Set<Author>();  
public DbSet<Category> Categories => Set<Category>();  
}  
  
dotnet ef migrations add InitialCreate  
dotnet ef database update
```

Practice — do these before the project

- Create a small standalone EF Core console/API project with just one entity to get comfortable with the migration commands before touching the real Library project.
- Insert and retrieve a few records using `context.Books.Add(...)` and `await context.SaveChangesAsync()`.
- Update one record and delete another using EF Core (not raw SQL).
- Write one filtered LINQ query against the DbSet, e.g. `context.Books.Where(b => b.AuthorId == 2).ToListAsync()`.

Git Checkpoint

Commit after the migration is created and again after it's applied, so the history shows the steps clearly:
`git commit -m "feat: add EF Core models and DbContext"` then `git commit -m "chore: add InitialCreate migration"`

PART C — API + DATABASE INTEGRATION

What to learn

- How to replace the temporary in-memory `List<Book>` repository from Week 2 with one that uses `LibraryDbContext` instead — the Controller and Service layers should barely change, which is the whole point of the layered architecture from Week 2.
- Handling database errors gracefully (e.g. record not found should return 404, not a raw exception).
- Why configuration (connection strings, keys) belongs in `appsettings.json` / environment variables, never hard-coded in a class file.

How to do it

In your `BookRepository` class, inject `LibraryDbContext` through the constructor (Dependency Injection), and replace each method's in-memory list logic with the equivalent EF Core call.

```
public class BookRepository : IBookRepository
{
    private readonly LibraryDbContext _context;
    public BookRepository(LibraryDbContext context) => _context = context;

    public async Task<List<Book>> GetAllAsync() =>
        await _context.Books.Include(b => b.Author).ToListAsync();

    public async Task<Book?> GetByIdAsync(int id) =>
        await _context.Books.FindAsync(id);
}
```

Practice

- Swap the in-memory repository for the EF Core one and re-run every endpoint in Swagger — nothing about the URL or request/response shape should change from the outside.
- Try requesting a book ID that doesn't exist and confirm the API returns 404, not a 500 error.
- Add basic try/catch around the save operation and return a clear error message if it fails.

Git Checkpoint

```
git commit -m "feat: replace in-memory repository with EF Core-backed repository"
```

PART D — AUTHENTICATION CONCEPTS (UNDERSTAND FIRST, BUILD LATER)

Do not implement login yet — the goal this week is to understand the vocabulary correctly, since getting these terms confused is the most common beginner mistake. Full implementation happens in Week 4.

What to learn

- **Authentication** — proving who you are (logging in).
- **Authorization** — deciding what you're allowed to do once you're known (e.g. only an Admin can delete a book).
- **Password hashing** — passwords are never stored as plain text; a one-way hash is stored instead.
- **JWT (JSON Web Token)** — a signed token issued after login that the client sends with future requests instead of logging in again every time. It has three parts: header, payload (claims), and signature.
- **Claims** — pieces of information about the user embedded inside the token (e.g. user ID, role).
- **Role-based authorization** — restricting an endpoint to users whose token contains a specific role claim.

Practice (written, not code)

- Take any JWT example token and identify its three dot-separated parts. Decode the header and payload (not the signature) using an online base64 decoder and explain what you see.
- Write, in your own words, what would go wrong if passwords were stored as plain text and the database was ever leaked.
- Draw a simple sequence: Login request → server checks password hash → server issues JWT → client stores JWT → client sends JWT on the next request → server validates it.

Git Checkpoint

Save your written answers and diagram as a markdown file, e.g. `/docs/week3-auth-notes.md`, and commit: `git commit -m "docs: add JWT and auth concept notes"`

PART E — ANGULAR API INTEGRATION

What to learn

- **HttpClient** — Angular's built-in service for making HTTP requests to your ASP.NET Core API.
- **Observables** — HttpClient calls return an Observable, not a plain value; you `subscribe()` to it to get the response when it arrives, since the network call is asynchronous.
- **Loading states** — a boolean flag the UI uses to show a spinner/message while waiting for the response.
- **Error handling** — using `catchError` (or a `try/catch` with `async/await`) so a failed request doesn't crash the component.
- **Environment files** — keeping the API base URL in `environment.ts` instead of hard-coding it inside every service call.

How to do it

- 1 Enable HttpClient for the app (via `provideHttpClient()` in `app.config.ts`, or the equivalent module import).
- 2 In `BookService`, inject `HttpClient` and replace the mock-data methods from Week 2 with real HTTP calls to your running API.
- 3 Add an `isLoading` and an `errorMessage` field in the component and set them before/after the call.

```
// book.service.ts
constructor(private http: HttpClient) {}

getBooks(): Observable<Book[]> {
  return this.http.get<Book[]>(`${environment.apiUrl}/api/books`);
}

// book-list.component.ts
this.isLoading = true;
this.bookService.getBooks().subscribe({
  next: (books) => { this.books = books; this.isLoading = false; },
  error: (err) => { this.errorMessage = 'Could not load books.'; this.isLoading = false; }
});
```

Practice

- Connect the Book service to your real API (not mock data) and confirm the table renders live data.
- Turn off your backend and confirm the Angular error state shows correctly instead of a blank page.
- Add a simple loading spinner or "Loading..." text tied to `isLoading`.
- Submit a Reactive Form to POST a new book and refresh the list after a successful save.

Git Checkpoint

```
git commit -m "feat: connect Angular BookService to live API with loading and error states"
```

PART F — GIT & GITHUB WORKFLOW (NEW THIS WEEK)

From Week 3 onward, every task in this document is expected to be committed as it's completed, using a proper branch-and-Pull-Request workflow — not one giant commit dumped at the end of the week.

What to learn

- **Repository (repo)** — the project folder tracked by Git, containing the full history of changes.
- **Commit** — a saved snapshot of changes with a message explaining what and why.
- **Branch** — an independent line of work, so new/risky changes don't affect the stable `main` branch until they're reviewed and merged.
- **.gitignore** — a file listing what Git should never track (build output, secrets, dependencies).
- **Pull Request (PR)** — a request to merge a branch into `main`, used as a checkpoint to review the diff before it becomes part of the project.
- **Merge conflict** — when two branches changed the same lines and Git needs a human decision to combine them.
- **README.md** — the front page of a repository; explains what the project is and how to run it.

Commit message convention to use from now on

Prefix every commit with a type, so the history is easy to scan:

Prefix	Use it for
feat:	a new feature or capability
fix:	a bug fix
docs:	documentation or notes only
chore:	setup, config, migrations, dependencies
refactor:	restructuring code with no behavior change

How to do it — repository setup

- 1 Create (or continue) a repository named `library-internship-project` on GitHub.
- 2 Clone it locally: `git clone <repo-url>`
- 3 Add a `.gitignore` covering `.NET`, `Angular/Node`, and `Python` in one file so build output, `node_modules`, and virtual environments are never committed.
- 4 Write a `README.md` with: project overview, tech stack, how to run the backend, how to run the frontend, how to run the AI script, and a table of API endpoints.
- 5 Push the initial setup to `main` as your very first commit.

```
# .gitignore (excerpt)
bin/
obj/
appsettings.Development.json

node_modules/
```

```
dist/  
  
.venv/  
__pycache__/  
.env
```

How to do it — feature branch workflow

- 1 Before starting a new part of the work, create a branch: `git checkout -b feature/ef-core-setup`
- 2 Work and commit normally on that branch, in small steps, with prefixed messages.
- 3 Push the branch: `git push -u origin feature/ef-core-setup`
- 4 On GitHub, open a Pull Request from your branch into main. Write 2–3 sentences describing what changed and why.
- 5 Review your own diff line by line (or ask your co-intern to review it) before merging — this is a real habit used on every professional team.
- 6 Merge the Pull Request (Squash and merge is fine for a small team), then delete the branch.

Branches expected this week

- feature/relational-schema — Part A SQL scripts
- feature/ef-core-setup — Part B models, DbContext, migrations
- feature/api-db-integration — Part C repository swap
- feature/angular-http-integration — Part E Angular changes
- feature/ai-summary-script — Part G Python script

Never commit secrets

Connection strings, API keys, and `.env` files must never appear in a commit. If one is committed by accident, tell your mentor immediately — the key must be rotated (replaced), not just deleted from a later commit, since it still exists in the history.

PART G — AI & PYTHON FOUNDATIONS (KICKOFF)

This is the first hands-on AI work in the internship. It stays intentionally small and separate from the .NET/Angular project this week — real integration between them starts in Week 6, after prompt engineering, embeddings, and LangChain are covered in Weeks 4–5.

What to learn

- **AI / ML / Deep Learning / NLP / Generative AI** — conceptual definitions and how they relate (each is a narrower idea inside the one before it).
- **LLM (Large Language Model)** — a model trained to predict the next piece of text; this is what powers ChatGPT/Claude-style APIs.
- **Training vs. inference** — training is when the model learns from data (done once, by the provider); inference is each time you call the API and it generates a response.
- **Tokens** — the small chunks of text (roughly parts of words) the model actually processes; API usage and cost are measured in tokens.
- **Context window** — the maximum number of tokens the model can "see" at once, including your prompt and its own reply.
- **Hallucination** — when the model confidently states something false; it happens because the model is predicting plausible text, not looking facts up, unless it's explicitly given real data to work from.

How to do it — Python environment

- 1 Create a project folder, e.g. /ai-scripts, inside the same repository.
- 2 Create and activate a virtual environment so this project's packages don't clash with anything else on your machine.
- 3 Install the SDK and dotenv package.
- 4 Create a .env file holding your API key — this file must be listed in .gitignore and must never be committed.

```
python -m venv .venv
source .venv/bin/activate # on Windows: .venv\Scripts\activate
pip install anthropic python-dotenv

# .env (never commit this file)
ANTHROPIC_API_KEY=your_key_here

# main.py
import os
from dotenv import load_dotenv
import anthropic

load_dotenv()
client = anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=300,
    messages=[{"role": "user", "content": "Explain what a token is in one paragraph."}]
)
print(response.content[0].text)
```

Practice — do these before the project

- Run the script above and confirm you get a real response back.
- Change the prompt three different ways — a plain question, a direct instruction, and a role-based prompt ("You are a strict librarian who only answers in one sentence...") — and write down how the output changed for each.
- Deliberately ask the model something obscure or made-up and observe whether it hallucinates an answer instead of saying it doesn't know.

Git Checkpoint

```
git commit -m "feat: add first AI/Python script with .env-based key loading"  
— double check .env is not staged before committing.
```

WEEK 3 PROJECT — DATABASE-BACKED LIBRARY API + ANGULAR CLIENT + FIRST AI SCRIPT

Continue the Week 2 Library project — do not start a new one. Work through the steps below in order; each step maps to a Git branch and Pull Request from Part F.

- 1 **Schema:** Create Book, Author, and Category tables as EF Core entities with the relationships designed in Part A. Branch: `feature/relational-schema`.
- 2 **EF Core setup:** Configure SQL Server and EF Core, create the first migration, update the database. Branch: `feature/ef-core-setup`.
- 3 **Repository swap:** Replace the in-memory repository from Week 2 with the EF Core-backed one, keeping the Controller and Service layers unchanged where possible. Branch: `feature/api-db-integration`.
- 4 **Verify persistence:** Test full CRUD with Postman — restart the API and confirm the data is still there.
- 5 **Angular integration:** Connect the Angular frontend to the real, running API. Add loading and error handling. Branch: `feature/angular-http-integration`.
- 6 **Login foundation:** Add a very small User/Login table and endpoint skeleton only — no real JWT issuing yet, that's Week 4. This just proves you understand where auth will plug in.
- 7 **AI script:** Write a standalone Python script that takes a book title and description and asks the LLM to generate a one-paragraph summary or suggest a genre. Print the result to the console — it does not need to be called from the API yet. Branch: `feature/ai-summary-script`.
- 8 **Documentation:** Update the README with the final data flow: Angular → API → Service → Repository → EF Core → SQL Server, plus a short "AI Track" section describing the standalone script and how to run it.
- 9 **Merge everything:** Make sure every feature branch has an individual Pull Request, reviewed and merged into main.
- 10 **Tag the milestone:** Once everything is merged, tag the commit: `git tag -a v0.3-week3 -m "Week 3: EF Core, Angular integration, first AI script"` then `git push origin v0.3-week3`.

Why the AI script stays separate this week

Wiring AI into the .NET API before prompt engineering, structured outputs, and error handling for LLM calls are covered (Week 4) would mean building on shaky ground. The standalone script proves the concept works; real integration comes once the AI track catches up.

GITHUB TASK CHECKLIST — WEEK 3

This checklist is graded separately from the code itself. A mentor should be able to open the repository and see this workflow without being told about it.

- ■ Repository exists with a clear name and is accessible to the mentor.
- ■ .gitignore covers .NET, Angular/Node, and Python — no bin/, obj/, node_modules/, .venv/, or .env ever appears in the history.
- ■ README.md includes: project overview, tech stack, backend run instructions, frontend run instructions, AI script run instructions, and an endpoints table.
- ■ At least 5 feature branches used this week, matching the list in Part F.
- ■ Commit messages use the feat: / fix: / docs: / chore: convention.
- ■ At least one Pull Request has a written description (not left blank) and was reviewed before merging.
- ■ No API keys, passwords, or connection strings appear anywhere in the commit history.
- ■ Repository is tagged v0.3-week3 after everything is merged.

What Candidates Should Be Able to Do After Week 3

- Design two or more related tables with correct primary/foreign keys and write a JOIN across them.
- Explain what EF Core, DbContext, DbSet, and a migration are, and use them to connect a real API to SQL Server.
- Explain the difference between authentication and authorization, and describe what a JWT contains.
- Connect an Angular service to a live backend using HttpClient, with loading and error states handled.
- Create feature branches, write proper commit messages, open a Pull Request, and merge it — without a mentor walking them through each command.
- Run a Python script that calls an LLM API safely, with the key loaded from a .env file, and explain what a token, a context window, and a hallucination are.

Helping Material

- EF Core overview: <https://learn.microsoft.com/en-us/ef/core/>
- EF Core getting started: <https://learn.microsoft.com/en-us/ef/core/get-started/overview/first-app>
- ASP.NET Core authentication & authorization: <https://learn.microsoft.com/en-us/aspnet/core/security/>
- Angular HTTP client guide: <https://angular.dev/guide/http>
- Git basics (branching): <https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell>
- GitHub Pull Request docs: <https://docs.github.com/en/pull-requests>
- Conventional Commits: <https://www.conventionalcommits.org/>
- Python venv docs: <https://docs.python.org/3/library/venv.html>
- python-dotenv: <https://pypi.org/project/python-dotenv/>
- Anthropic API quickstart: <https://docs.claude.com/en/docs/get-started>

Week 3 Questions (Check Stage — answer without notes)

- What is the difference between a primary key and a foreign key?
- What does a JOIN actually do, and why can't you get the same result from one table alone?
- What is DbContext, and what does a DbSet represent?
- What is a migration, and why is it safer than manually editing the database?
- Why should database credentials and API keys live outside the source code?
- What is the difference between authentication and authorization?
- What is a JWT made of, at a high level?
- How does Angular's HttpClient know a request finished — what role does an Observable play?
- What is a Git branch, and why not just commit everything directly to main?
- What does a Pull Request actually check for, that a direct commit doesn't?
- What is a token in the context of an LLM, and what is a context window?
- Why does a model hallucinate, and how could giving it real data reduce that?

Suggested Mentor Review — Week 3

- Ask the candidate to open the GitHub repository live and walk through the commit history and at least one merged Pull Request, explaining what changed and why.
- Ask them to add a new field to the Book entity live (e.g. PublishedYear) and run a new migration in front of you.
- Ask them to explain, without opening the code, what happens end-to-end when a user clicks "Save" on the Angular book form.
- Ask them to intentionally break the API connection and show that Angular's error state handles it gracefully.
- Ask them to run their AI script live and explain, in their own words, what a token and a hallucination are.
- Ask one candidate to review the other's Pull Request on the spot and describe what they'd comment on.